Tabular Reinforcement Learning

Sisay Tadesse Ketema [* 1]

Abstract

In this paper, we compare a Dynamic Programming method, specifically Q-value Iteration that solves the planning problem, against Model-free Reinforcement methods that solve the Reinforcement learning problem. We discuss the limitations of Dynamic programming, and the various aspects of Model-free tabular Reinforcement Learning Algorithms like SARSA, MonteCarlo Reinforcement learning, and Q-learning.

1. Introduction

Learning from interactions is a fundamental idea underlying nearly all theories of learning and intelligence (1). Every learning being gets the first understanding of its world not through instruction, but through directly interacting with its environment, gradually testing, observing, and adapting for certain goal. A newly born deer taking its first steps minutes after being born. A baby elephant using its trunk to fetch water for its first drink. All these are examples of learning through interaction with the environment, where each living being learns the skills that decide its survival.

Reinforcement Learning (RL) is a machine learning paradigm in which agents learn what to do from interaction. The main goal of RL agents is to maximize the numerical reward signals that result from interacting with the environment. Generally, RL is a computational approach to learning from interactions with the environment.

Tabular Reinforcement Learning is RL in its simplest form. The state and action spaces are small enough that the expected returns from states $v(s)$ and state-action pairs $q(s, a)$ can be represented using tables. These are the values that quantify the rewards an agent gains. The Methods used to solve Tabular RL are usually able to find exact solutions: optimal policies $\pi_*(s, a)$ that pick the optimal action a given state s. A good example is Dynamic programming (DP): Given a model of the environment as a Markov decision process, DP methods can determine the exact optimal policy. When the environment model is not known, which is usually the case, model-free RL algorithms are used to approximate optimal value functions and find optimal policies.

2. Theory

An environment can be defined using the following concepts. Possible states, actions, next states, and rewards gained from actions taken from a given state. The sequential nature of such environments can be modeled with Markov decision processes (MDPs). The MDP gives the joint probability distribution over the next possible state and reward, given a current state and action.

$$p(s', r|s, a) = Pr\{S_t = s', R_t = r|S_{t-1} = s, A_{t-1} = a\} \tag{1}$$

for all $s', s \in S$, $r \in R$, and $a \in A(s)$, the function p defines the dynamics of the MDP.

The goal of the agent is to maximize the cumulative rewards it receives from this sequential decision making. For a sequence of rewards $R_{t+1}, R_{t+2}, R_{t+3}, ...$ that come after timestep t, the cumulative reward is defined with the Return G_t.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + ... \tag{2}$$

where γ is a discount-rate, $0 \leq \gamma \leq 1$ for rewards that come later in the sequence.

The quality of a state for an agent is described using *value functions*. For an agent following a policy $\pi(a|s)$, the value function represents the expected return obtained when starting from a state $s \in S$ and continuing to take actions according to the policy. Since the policy defines a probability distribution over actions given states, the state value and the state-action value functions are defined for the policy:

$$v_\pi(s) = \mathbb{E}[G_t|S_t = s] \tag{3}$$

$$q_\pi(s, a) = \mathbb{E}[G_t|S_t = s, A_t = a] \tag{4}$$

Value functions satisfy a recursive property. The value function of a state at timestep t is a function of the value function at timestep $t + 1$. The bellman equation defines this recursion as:

$$v_\pi(S_t) = \mathbb{E}_\pi \left[R_{t+1} + v_\pi(S_{t+1}) \mid S_t = s\right] \tag{5}$$

The value functions can be estimated from experience or can be determined with dynamic programming depending on the availability of a complete MDP for the environment. These two ways of solving for value functions give rise to model-based and model-free tabular RL algorithms.

Algorithm 1 Q-value Iteration (Dynamic Programming)

Input: Threshold $\eta \in \mathbb{R}^+$
Result: Optimal value function $Q^*(s,a)$
Initialization: $\hat{Q}(s,a) \leftarrow 0,\ \forall s \in S, a \in A$
repeat
  $\Delta \leftarrow 0$
  for $s \in S$ **do**
    for $a \in A$ **do**
      $x \leftarrow Q(s,a)$
      $Q(s,a) \leftarrow \sum_{s'} P(s'|s,a)\left[R(s,a,s') + \gamma \max_{a'} Q(s',a')\right]$
      $\Delta \leftarrow \max(\Delta, |x - Q(s,a)|)$
    end for
  end for
until $\Delta < \eta$
$Q^*(s,a) = \hat{Q}(s,a)$
$\pi^*(s) = argmax_a Q^*(s,a)$
Return: $Q^*(s,a)$ and $\pi^*(s)$

2.1. Dynamic programming: Q-value Iteration

Given a finite set of states, actions, and rewards together with the model of the dynamics of the environment, DP can determine the optimal policy by using the Bellman Equation as an update rule.

$$v_{k+1}(s) = \mathbb{E}_\pi \left[R_{t+1} + v_k(S_{t+1}) \mid S_t = s\right] \tag{6}$$

By iteratively sweeping through all states and updating their value functions with the estimates from a step before and the immediate reward, DP guaranties the values goes to the optimal value function as k goes to ∞. After that the optimal policy can be found by greedily acting on the optimal value functions from one step early. The Q-value iteration, a DP method that updates the state-value actions based on the Bellman's optimality equation, is given in Algorithm 1.

2.2. Model-free Reinforcement Learning

Environments in most cases do not have a complete model, or a finite set of actions, or states. In these scenarios, implementing dynamic programming with full sweep update for every Q(s,a) fails. There might even be sets of states and actions that will not be discovered all together. To tackle this problem the agent need to balance it exploration and exploitation tendencies.

2.2.1. Exploration and Exploitation

To make novel exploration possible a model-free agent can use the a softmax policy for choosing actions or ϵ greedy policy .

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|A(s)|}, & \text{if } a = \arg\max_{a'} Q(s,a') \\ \frac{\epsilon}{|A(s)|}, & \text{otherwise} \end{cases} \tag{7}$$

The ϵ-greedy policy balances exploration and exploitation by selecting the action with the highest estimated value with probability $1 - \epsilon$. With probability ϵ, the agent selects a random action to explore the environment.

The softmax policy selects actions probabilistically according to their estimated action-values. The temperature parameter τ controls the degree of exploration, with higher values leading to more uniform action selection.

$$\pi(a|s) = \frac{\exp\left(\frac{Q(s,a)}{\tau}\right)}{\sum_{a'} \exp\left(\frac{Q(s,a')}{\tau}\right)} \tag{8}$$

The Q-learning algorithm, implements an off policy update to state-action values seen in an episode, by nudging the q-values found by an exploring policy, in the direction that minimizes the error if the agent was acting greedily in the next states.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\right] \tag{9}$$

2.2.2. Back-up: On-policy versus off-policy target

In SARSA, the backup update uses the action that the agent actually follows under its current policy. After the agent observes the transition $(S_t, A_t, R_{t+1}, S_{t+1})$, it selects the next action A_{t+1} using the same policy that is used for interaction with the environment, for example an ϵ-greedy policy. The value of the state–action pair $Q(S_t, A_t)$ is then updated using the reward received and the estimated value of the next state–action pair $Q(S_{t+1}, A_{t+1})$.

The key characteristic of SARSA is that the backup is *policy-consistent*, meaning the update reflects the behaviour of the policy that is actually being executed. Instead of using the maximum possible next action value, as in off-policy methods such as Q-learning, SARSA uses the value of the action that the current policy would take. As a result, the learning process evaluates and improves the same policy that generates the experience, making SARSA more sensitive to exploration and often producing safer behaviour in stochastic environments.

2.2.3. Back-up: Depth of target

Unlike Q-learning or SARSA, the implementation of n-step methods or Monte Carlo (MC) methods requires collecting a complete episode before updating the value estimates. An episode consists of a sequence of state-action-reward transitions, for example: $[S_1, A_1, R_2, S_2, A_2, R_3, \ldots, S_T, A_T]$, where S_T denotes the final state of the collected episode. Note that this state does not necessarily correspond to a terminal state, as the episode length T can be treated as a tunable hyperparameter.

While Q-learning and SARSA update their estimates after observing a single reward and bootstrapping from the next state, n-step Temporal-Difference methods—such as n-step Q-learning and Monte Carlo—accumulate the first n rewards before performing a bootstrap update based on the value estimate after n transitions.

3. Experiments

3.1. Environment

The stochastic Windy Grid world, a well known example from Reinforcement Learning: An Introduction (second edition) by Sutton and Barto, is adapted to test and compare our algorithms(1). It is a grid world with a total of 70 states, 7 rows, and 10 columns. A Stochastic wind acts on the agent upward, between columns 4 to 9, inclusive. A step in this world has a reward -1, unless that step takes the agent to a goal state.

3.2. Results

3.2.1. Dynamic Programming- Q value Iteration

Figure 4 illustrates how Q values change for different iteration steps. As the algorithm progress from the zero initialization of the q-values to the final stages, the reward at the goal location propagates to every state value, and in effect increasing the q-values. At the optimal value, the converged value of the starting location [0,3] is 83.7. This quantity is the expected return the agent can get if it follows the optimal policy from that state onward.

The average reward per time step under the optimal policy can be found using the following variables. q-value of starting state of value 83.7 , final reward of 100, and reward per step of -1. By first calculating the average steps to reach the goal as the difference between the final reward and the optimal value of the starting location, that is 16.4. And then average return of 83.7 to average steps to reach goal 16.4, the average reward per step is 5.10.

Updating the goal location with new location [6,2] , gives two noticeable results. The first difference is in the optimal value of the start state, which plummeted to 64.8 from the earlier value of 83.7. The second difference is in the way the agent behaves under the new optimal policy. The agent tends to go against the direction of the wind at the locations [3,1] and the neighboring locations in the same line to the right of [3,1]. Optimal behavior from the previous goal state tried to use the wind direction to its advantage.

Dynamic programming deals with goal states that terminate episodes, by modeling the transition in a way that the agent doesn't escape that goal state. The MDP models transitions out of the goal state to be zero.

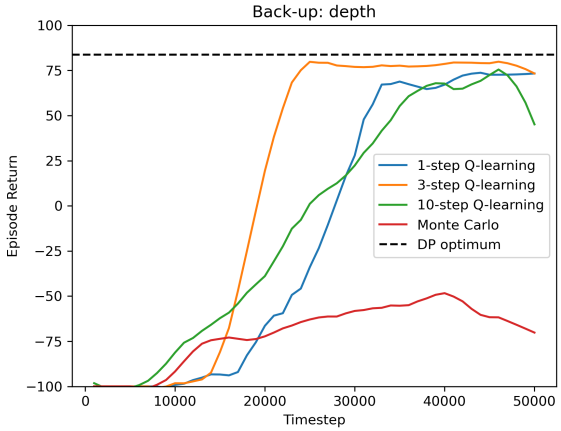


Figure 1. Comparison between different back-up depths. .

3.2.2. Comparison of exploration methods

From Figure 3 it can be seen both e-greedy and softmax policies fairly approximate the dynamic programming. Higher values for ϵ tend to make the curve converge early, but very unstable too. The Softmax policy achieves the optimal when the temperature is set to 1, meaning actions are selected according to their relative Q-values, providing a balanced trade-off between exploration and exploitation.

3.2.3. Comparison of Back-up: On-policy versus off-policy target

When both back-up methods are used with smaller learning rate of 0.03, the experiment result shows the curves converge very slowly. and the same is true for higher learning rate of 0.3 both converge faster. The optimal learning rate of both back-up methods is 0.1 where they reach the level of the dynamic programming optimal Q-value for the start state figure 2.

3.2.4. Comparison of Back-up: Depth of target

N-step q-learning converges very fast for back depth of 3 steps, and performs better than the 10 step and 3 step q-learning algorithms. The 10 step q-learning algorithms is unstable compared to the other two. Monte-carlo method with a full back up until episode termination performed the worst.

References

[1] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2020.

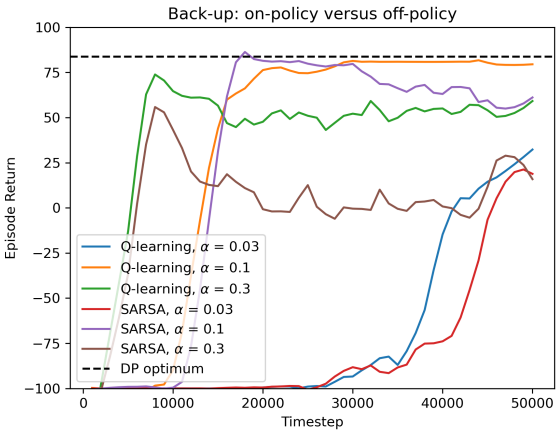


Figure 2. Comparison between back-up methods. Q-learning is an off-policy method that updates the current policy using another greedy policy's estimate. SARSA is an on-policy method that incrementally updates from earlier time steps by the chosen policy. Result showing three different learning rates used for both backup methods.

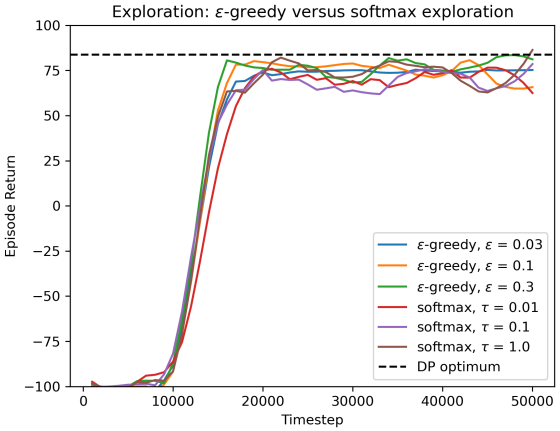
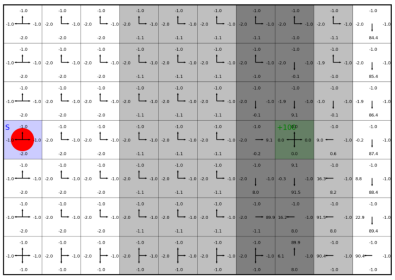
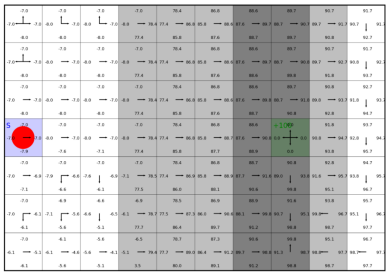


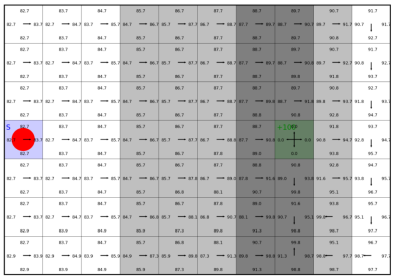
Figure 3. Comparison of softmax and ϵ greedy exploration policies, under different hyperparameter of temperature τ for softmax, and different ϵ for ϵ-greedy policies. The dashed line is the optimal value for the starting state under dynamic programming.



(a) Beginning iteration



(b) Middle iteration



(c) Final iteration

Figure 4. Evolution of Q-values in the stochastic windy grid world. a) Q values after one-step sweep update: results show for most state-action values the step reward is set, -1 for most, and above 90 for a another state below to the goal b) Q values mid-iteration - as the update progresses the states very far from the goal decrease rapidly, while those close to the goal increase C) optimal Q values reached